

Modeling Computation

R Pruim

2019-04-30

Contents

Preface	2
1 Alphabets, Languages, and Grammars	1
1.1 Terminology	1
1.2 (Phrase-Structure) Grammars	1
2 Types of Grammars	3
2.1 Backus-Naur Form (BNF) for Context Free Grammars	3
3 Finite State Automata	5
3.1 Graph Representation	5
3.2 Extended transition function	5
3.3 Language Recognition	6
3.4 Equivalent Automata and Grammars	7
3.5 Nondeterministic Automata	7
4 Regular Expressions and Languages	9
4.1 Operations on Languages	9
4.2 Regular Expressions	9
4.3 Recognizing Regular Languages with NFAs	10
4.4 Solution to problem 4.3.9	11
5 Regular Grammars and Automata	12
5.1 Grammars to Automata	12
5.2 Automata to Grammars	13
5.3 Some Review	14
6 Regular Languages	15
6.1 Equivalent Definitions	15
6.2 A language that is not regular	15
6.3 Solutions	16
7 DFA/NFA to Regular Expression	18
7.1 GNFA (Generalized NFA)	18
7.2 Algorithm for converting DFA/NFA to Regular Expression	18
7.3 Examples	18
7.4 Review problems	19
8 Regular Expression in Python	19
8.1 Getting your python environment working	19

Preface

These exercises were assembled to accompany a Discrete Mathematics course at Calvin College.

1 Alphabets, Languages, and Grammars

1.1 Terminology

- An **alphabet** (or **vocabulary**) is just a finite, non-empty set. Its elements are called **letters** or **symbols**.
- A **word** (or **string**) is a finite sequence of symbols.
- A word consisting of no symbols is called the **empty word** and denoted λ . (Think "".)
- The set of all words using exactly n symbols from V (repetition allowed) is denoted V^n .
- The set of all words using symbols in alphabet V is denoted V^* . (So $V^* = V^0 \cup V^1 \cup V^2 \dots$)
- A **language** over V is a subset of V^* .

Exercises

1. Let $V = \{0, 1\}$. What is V^0 ? What is V^2 ? What is V^* ?
2. Let $V = \{2\}$. What is V^0 ? What is V^2 ? What is V^* ?
3. Can $\emptyset$ (empty set) be a letter? an alphabet? a word? a language?

1.2 (Phrase-Structure) Grammars

A phrase-structure grammar consists of

- **alphabet:** V
- **start symbol:** $S \in V$
- **terminal symbols:** $T \subset V$
 - $N = V - T$ is the set of **nonterminal symbols**
- **production rules:** $P \subseteq (V^* - N^*) \times V^*$
 - usually write elements of P as $u \rightarrow v$ rather than (u, v) .
 - $V^* - N^*$ says that u must contain at least one nonterminal.
 - v can be any combination of terminals and nonterminals (including the empty string).

Examples

Note: in each of these examples, capital letters are used for nonterminals and lower case letters or digits for terminals. That makes it easy to remember, but it is not a requirement of the definition of a grammar.

Grammar 1 (G_1): alphabet: $\{a, b, A, B, S\}$, terminals: $\{a, b\}$, start symbol: S , production rules:

- $S \rightarrow ABa$
- $A \rightarrow BB$
- $B \rightarrow ab$
- $AB \rightarrow b$

Grammar 2 (G_2): alphabet: $\{S, A, a, b\}$, terminals: $\{a, b\}$, start symbol: S , production rules:

- $S \rightarrow aA$
- $S \rightarrow b$
- $A \rightarrow aa$

Grammar 3 (G_3): alphabet: $\{S, 0, 1\}$, terminals: $\{0, 1\}$, start symbol: S , production rules:

- $S \rightarrow 11S$
- $S \rightarrow 0$

Grammar 4 (G_4): alphabet: $\{S, A, B, C, a, b, c\}$, terminals: $\{a, b, c\}$, start symbol: S , production rules:

- $S \rightarrow AB$
- $A \rightarrow Ca$
- $B \rightarrow Ba$
- $B \rightarrow Cb$
- $B \rightarrow b$
- $C \rightarrow cb$
- $C \rightarrow b$

1.2.1 Derivations and Languages

The rules of a grammar are used to derive strings of terminals (elements of T^*) as follows.

- If $w_0 \rightarrow w_1$ is a rule, then $lw_0r \Rightarrow lw_1r$ for any strings l and r .
- $a \xRightarrow{*} b$ is defined recursively. $a \xRightarrow{*} b$ if either
 - $a \Rightarrow b$, or
 - $\exists c (a \Rightarrow c \wedge c \xRightarrow{*} b)$

Basically the production rules are “rewrite rules” that allow us to replace the left side of the rule with the right side. A derivation is a sequence of rewrites.

The language of a grammar G is denoted $L(G)$ and contains all strings of terminals that can be derived from the start symbol:

$$L(G) = \{w \in T^* \mid S \xRightarrow{*} w\}$$

1.2.1.1 Exercises

4. Show that using Grammar 1, $ABa \xRightarrow{*} abababa$.
5. Is $abababa \in L(G_1)$, the language generated by Grammar 1?
6. What is $L(G_2)$?
7. What is $L(G_3)$?
8. Generate several words using G_4 .
9. Create a grammar G_5 such that $L(G_5) = \{0^n 1^n \mid n = 0, 1, 2, \dots\}$
10. Create a grammar G_6 such that $L(G_6) = \{0^n 1^n \mid n \in \mathbb{Z}^+\}$
11. Create a grammar G_7 such that $L(G_7) = \{0^m 1^n \mid m, n \in \mathbb{N}\}$

2 Types of Grammars

Type 0: no restrictions

Type 1 (context sensitive): only two types of rules allowed

- $lAr \rightarrow lwr$ where
 - $l, r \in V^*$
 - $A \in N$
 - $w \in V^+$ [That is, $w \neq \lambda$.]
 - shorthand: `[left] [nonterminal] [right] → [left] [non-empty] [right]`
- $S \rightarrow \lambda$
 - shorthand: `[start] → [empty string]`
 - If the rule is used, the the start symbol may not appear on the right side of any rule, so other rules become `[left] [non-empty without start symbol] [right]`.

Type 2 (context free): only one type of rule allowed

- $A \rightarrow w$ where
 - A is a single nonterminal symbol
 - $w \in V^*$ (so no restriction here)
- shorthand: `[nonterminal] → [any string]`

Type 3 (regular): Three types of rules allowed

- `[nonterminal] → [terminal] [non terminal]` (Example: $A \rightarrow bC$)
- `[nonterminal] → [terminal]` (Example: $A \rightarrow b$)
- `[start] → [empty string]` (Example: $S \rightarrow \lambda$)

A regular/context free/context sensitive language is a language that can be generated by a regular/context free/context sensitive grammar.

1. For each grammar we have seen so far, determine whether it is regular, context free, context sensitive. (Some grammars will be more than one. All grammars are type 0.)
2. True or false: Every grammar of type n is a grammar of type $n - 1$.

2.1 Backus-Naur Form (BNF) for Context Free Grammars

Shorthand notation for type 2 grammars.

- nonterminals denoted with $\langle \rangle$.
- $\rightarrow$ written as `::=`
- All rules with same nonterminal on left written together with the list of possible right sides separated by `|`.

Example: If we have production rules $A \rightarrow Aa$, $A \rightarrow a$, and $A \rightarrow AB$, we will write

`<A> ::= <A>a | a | <A><B>`

BNF for ALGOL 60 identifier

```
<identifier> ::= <letter> | <identifier><letter> | <identifier><digit>
<letter> ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF for signed integer

```
<signed integer> ::= <sign><integer>
<sign> ::= + | -
<integer> ::= <digit> | <digit><integer>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF specification for Java

You can find BNF for various programming languages online. For example: <https://users-cs.au.dk/amoeller/RegAut/JavaBNF.html> has a BNF specification for Java.

Exercises

3. Why does BNF notation only work for context free grammars?
4. For each context free grammar we have seen, give its BNF representation.

3 Finite State Automata

A **finite state automaton** consists of the following:

- a finite set of **states**: S
- a finite **input alphabet**: I
- a **start state**: $s_0 \in S$.
- a set of **final states**: $F \subseteq S$
- a **transition function**: $f : S \times I \rightarrow S$

Example (M_0) states: $\{A, B, C, D\}$; input alphabet: $\{0, 1\}$; start state: A ; final states: $\{A, D\}$; transition function described in table below

M_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	A	B	A	C	A	A	C	B

3.1 Graph Representation

1. It is often easier to visualize what is going on if we represent an automaton with a graph. Create a labeled, directed graph with the following properties:
 - There is a vertex for each state, labeled with the state. (Draw this as a circle with the state inside the circle.)
 - There is an edge from state s to state t labeled with letter x if and only if $f(s, x) = t$
 - Final states are circled a second time. (We could use shape or color or something else to make it clear which states are final states, but double circling is easy.)
 - An extra arrow (not coming from any state) points to the start state. This isn't really an edge in the graph, just an extra bit of labeling.

3.2 Extended transition function

We can extend the transition function to $f^* : S \times I^* \rightarrow S$ with the following recursive definition:

- $f^*(s, \lambda) = s$ for any state s
 - $f^*(s, xa) = f(f^*(s, x)a)$ for any $x \in I^*$ and $a \in I$
2. Use this definition to determine the following.
 - a. $f^*(B, \lambda)$
 - b. $f^*(B, 0)$
 - c. $f^*(B, 010)$
 - d. $f^*(A, 101)$
 - e. $f^*(A, 1011)$

3.3 Language Recognition

For any automaton M with transition function f , start state s and final states F , the language recognized by M (written $L(M)$) is defined as follows:

$$x \in L(M) \iff f^*(s, x) \in F$$

3. For each string x below, compute $f^*(A, x)$. Which of these strings are in $L(M_0)$?
(We will say that such strings are **accepted by** M_0 .)

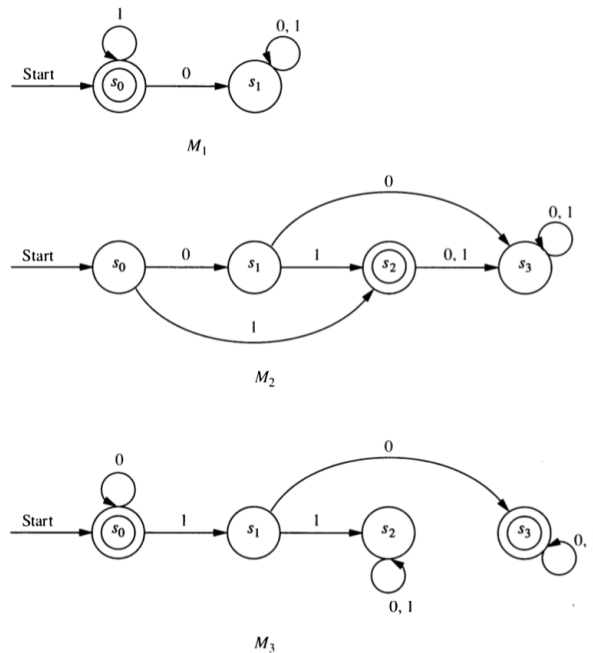
- a. λ
- b. 0
- c. 1
- d. 010
- e. 1011

Here are three automata:

4. For each of the machines M_1, M_2, M_3 , compute

- a. $f^*(s_0, 10)$
- b. $f^*(s_0, 1011)$
- c. $f^*(s_1, 1011)$

5. For each of the machines M_1, M_2, M_3 , determine the language recognized.



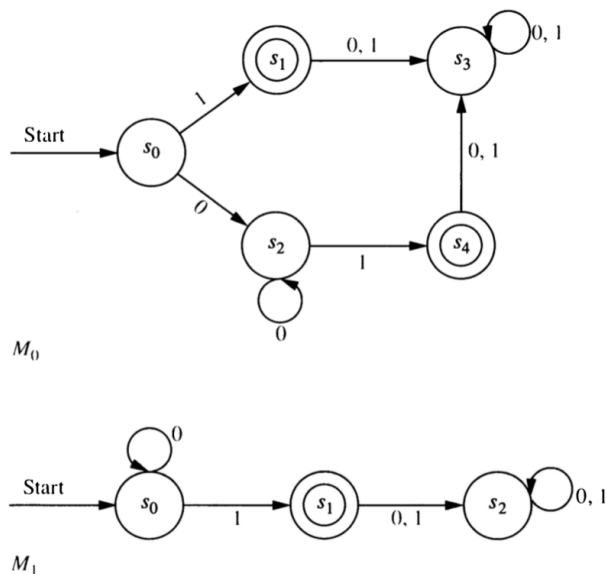
6. Create automata that recognize each of the following languages.

- a. The set of bitstrings that begin with two 0's
- b. The set of bitstrings that contain exactly two 0's
- c. The set of bitstrings that contain at least two 0's
- d. The set of bitstrings that contain two consecutive 0's (anywhere in the string)
- e. The set of bitstrings that do not contain two consecutive 0's anywhere
- f. The set of bitstrings that end with two 0's

3.4 Equivalent Automata and Grammars

We will say that two automata M and N are equivalent if $L(M) = L(N)$, that is, if they recognize the same language. Similarly, two grammars are equivalent if they generate the same language.

7. What must you do to show that two automata are **not** equivalent?
8. What must you do to show that two automata **are** equivalent.
9. Are the two automata represented below equivalent? Explain.



3.5 Nondeterministic Automata

The finite state automata we have seen so far are often called **deterministic finite-state automata** or DFAs. There is a generalization called a **non-deterministic finite-state automaton** or NFA. The only difference is how the transition function is specified. For an NFA, the transition function has the form

$$f : S \times I^* \rightarrow P(S)$$

where S is the set of states, I is the input alphabet, and $P(S)$ is the powerset of S . (The powerset of S is the set of all subsets of S . So the transition function for an NFA returns a *set* of states.)

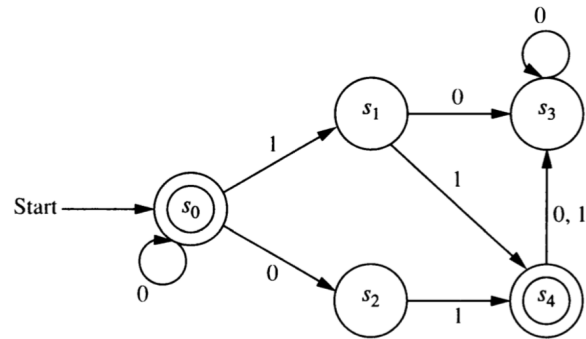
Here is a tabular description of the transition function for an NFA N_0 with start state A and final states C and D.

N_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	$\{A, B\}$	$\{D\}$	$\{A\}$	$\{B, D\}$	$\{\}$	$\{A, C\}$	$\{A, B, C\}$	$\{B\}$

10. Draw the graph representation. How will the graph of an NFA differ from the graph for a DFA?
11. How do we define the language recognized by an NFA? [Check your answer before proceeding.]
12. Which of these strings are accepted by N_0 ?

- a. 01
- b. 011
- c. 00
- d. 0000
- e. λ
- f. 010101

Here is the graph of an NFA N_1 .



13. Make a table showing the transition function for N_1 .
14. Which of the following strings are accepted by N_1 ?
 - a. λ
 - b. 00
 - c. 01
 - d. 010
 - e. 011
 - f. 001
15. What is $L(N_1)$?
16. Create a DFA that recognizes $L(N_1)$.
17. Given an NFA N , is it always possible to create a DFA M that recognizes the same language? If so, explain how. If not, provide an example N and explain why $L(N)$ cannot be recognized by a DFA.

4 Regular Expressions and Languages

4.1 Operations on Languages

Since languages are sets, we can use set operations like $\cup$ and $\cap$ to create new languages. But there are some additional operations that we will use.

- concatenation of strings: xy is the concatenation of strings x and y (x followed by y)
- concatenation of sets: $AB = \{xy \mid x \in A \wedge y \in B\}$
- power: $A^0 = \emptyset$; $A^1 = A$; $A^{n+1} = A^n A$. (Power is iterated concatenation.)
- Kleene star: $A^* = \bigcup_{n=0}^{\infty} A^n$

1. Which of these strings belong to $\{0,01\}^*$?
 - a. 01001
 - b. 10110
 - c. 00010
2. Which of these strings belong to $\{101,111,11\}^*$?
 - a. 1010111
 - b. 1011011
 - c. 1110111
 - d. 11110111

4.2 Regular Expressions

4.2.1 Syntax

A regular expression over an input alphabet I is defined recursively.

- $\emptyset$ and λ are regular expressions.
- x is a regular expression for each $i \in I$.
- If A and B are regular expressions, then
 - A^* is a regular expression
 - $(A \cup B)$ is a regular expression
 - (AB) is a regular expression

4.2.2 Semantics

Each regular expression represents a language as follows.

- $\emptyset$ represents the empty language: $\emptyset$
- λ represents language that contains only the empty string: $\{\lambda\}$
- For each $x \in I$, x represents the language containing only x : $\{x\}$
- If A and B are regular expressions representing languages A and B , then
 - (AB) represents AB
 - $(A \cup B)$ represents $A \cup B$ [Note: This is often written $(A|B)$ in programming languages.]
 - A^* represents A^* [Note: This is often written as A^* in programming languages.]

The languages represented by regular expressions are called **regular languages** (how creative).

3. In Rosen 7, the definition of regular expression is missing boldface in a couple places. Find them.

The *regular expressions* over a set I are defined recursively by:

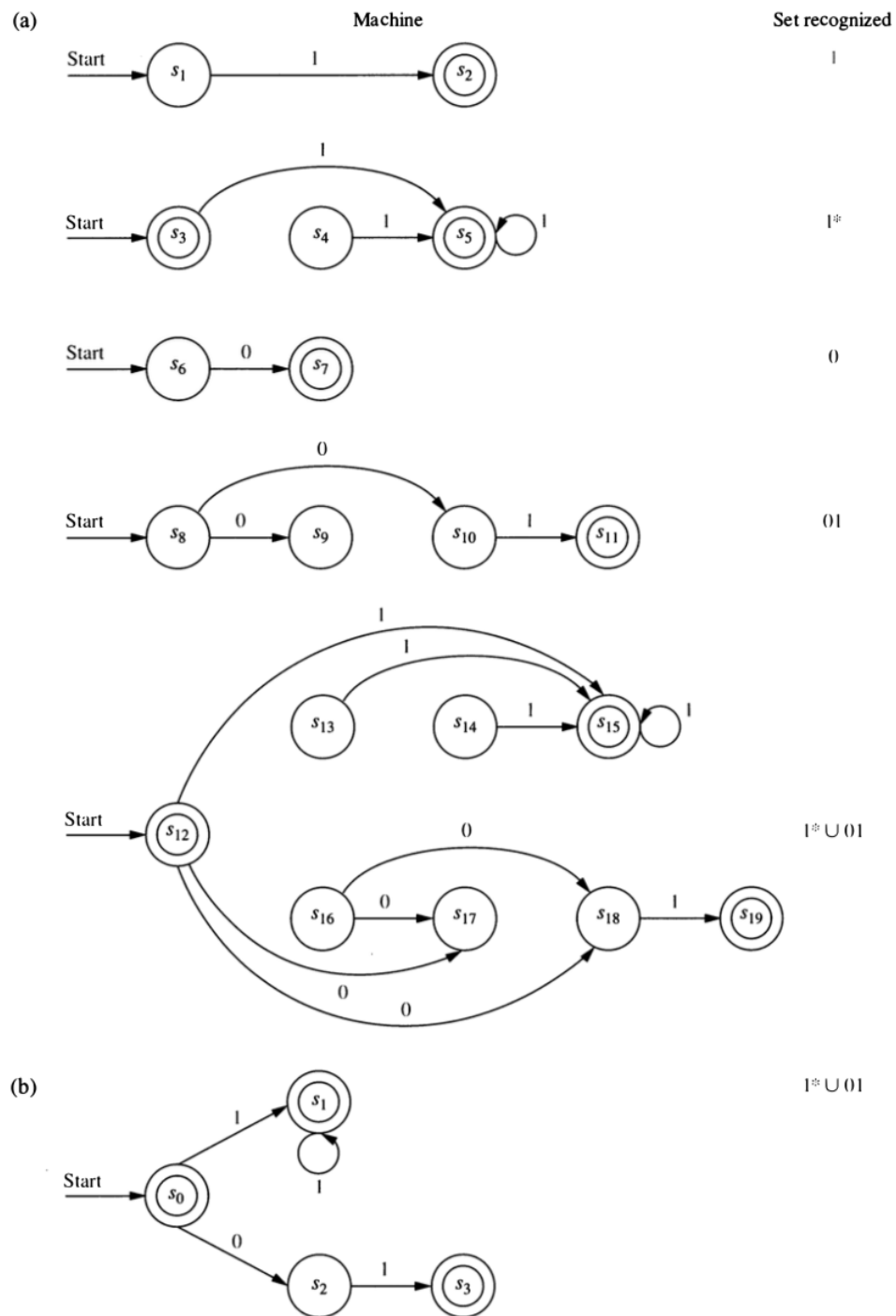
- the symbol $\emptyset$ is a regular expression;
- the symbol λ is a regular expression;
- the symbol x is a regular expression whenever $x \in I$;
- the symbols $(\mathbf{AB})$, $(\mathbf{A} \cup \mathbf{B})$, and $\mathbf{A}^*$ are regular expressions whenever $\mathbf{A}$ and $\mathbf{B}$ are regular expressions.

4. What is the difference between $\mathbf{1}$ and 1 ? between $\boldsymbol{\lambda}$ and λ ? between $\boldsymbol{\emptyset}$ and $\emptyset$?
5. Describe in words the languages represented by each of these regular expressions:
- a. $\mathbf{10}^*$ b. $(\mathbf{10})^*$ c. $\mathbf{0} \cup \mathbf{01}$ d. $\mathbf{0}^* \cup \mathbf{01}$ e. $\mathbf{0}(\mathbf{0} \cup \mathbf{1})^*$ f. $(\mathbf{0} \cup \mathbf{01})^*$
6. Can each of the languages above be recognized by an NFA? a DFA?
7. What would we need to do to show that every regular language can be recognized by an NFA? Give it a try and see how far you get. Which parts are easy? Which are trickier? Why?

4.3 Recognizing Regular Languages with NFAs

1. Show that the empty language ($\emptyset$) is recognized by an NFA.
2. Show that the language $\{\lambda\}$ is recognized by an NFA.
3. Show that if $a \in I$, then the language $\{a\}$ is recognized by an NFA.
4. Show that if A and B are each recognized by an NFA, then AB is, too.
 [Hint: Let N_A and N_B be the NFAs that recognize A and B . How can you use them to build a new NFA that recognizes AB ?]
5. Show that if A and B are each recognized by an NFA, then $A \cup B$ is, too.
 [Hint: Let N_A and N_B be the NFAs that recognize A and B . How can you use them to build a new NFA that recognizes $A \cup B$?]
6. Show that if A is recognized by an NFA, then A^* is, too.
 [Hint: Let N_A be the NFA that recognizes A . How can you use it to build a new NFA that recognizes $A \cup B$?]
7. Explain how 1–6 above show that every regular language is recognized by an NFA.
8. Explain how 7 implies that every regular language is recognized by a DFA.
9. The method just outlined is automatic (you could fairly easily write a computer program to do the translation from regular expression to NFA), and it provides a proof that all regular languages can be recognized by an NFA. But it might produce an NFA with more states than is minimally required. Create an NFA that recognizes $\mathbf{1}^* \cup \mathbf{01}$. Do this two ways:
 - a. Following the steps outlined in 1–6.
 - b. Any way you like, but using fewer states than in part a.
 - c. What is the smallest (fewest states) NFA you can find that recognizes $\mathbf{1}^* \cup \mathbf{01}$?
10. Show that if A is recognized by an NFA, then the complement of A ($\bar{A} = A^C$) is too.
 [Hint: There is an easy way and a hard way – use the easy way.]

4.4 Solution to problem 4.3.9



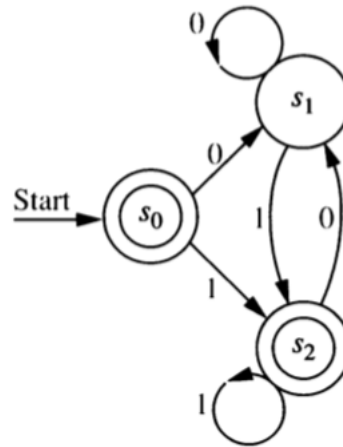
5 Regular Grammars and Automata

5.1 Grammars to Automata

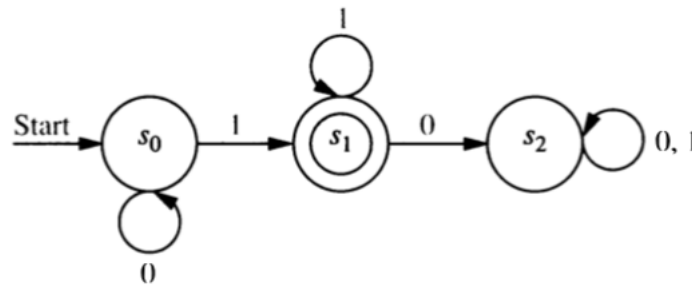
1. Construct a finite state automaton that recognizes the language generated by the following regular grammar.
 - terminals: 0 and 1
 - nonterminals: S and A
 - start symbol: S .
 - production rules: $S \rightarrow 1A$, $S \rightarrow 0$, $S \rightarrow \lambda$, $A \rightarrow 0A$, $A \rightarrow 1A$, $A \rightarrow 1$
2. Construct a finite state automaton that recognizes the language generated by the following regular grammar.
 - terminals: 0 and 1
 - nonterminals: S , A , B
 - start symbol: S .
 - production rules: $S \rightarrow 1A$, $S \rightarrow 0$, $A \rightarrow 0B$, $A \rightarrow 1A$, $A \rightarrow 1 B \rightarrow 0A$, $B \rightarrow 1B$, $B \rightarrow 0$
3. Explain how *any* regular grammar G can be converted into an NFA N such that $L(N) = L(G)$.
 - a. What is a regular grammar? (What sorts of production rules are allowed?)
 - b. What will the states of N be? How many will there be?
 - c. What state will be the start state?
 - d. Which states will be final (ie. accepting) states?
 - e. How does each rule in G get converted into a part of N ? [Go through each kind of rule a regular grammar may have.]

5.2 Automata to Grammars

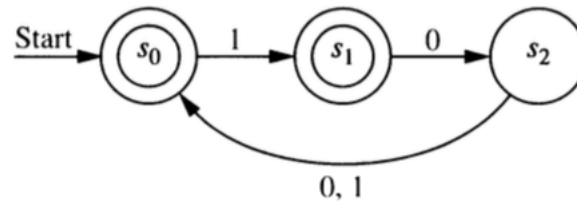
4. Show that every DFA or NFA is equivalent to a DFA or NFA with a “lonely start state”. A lonely start state is a start state that can never be returned to. (In terms of the graph, its in-degree is 0.)
5. Convert each of the automata in the next three problems into an automaton with a lonely start state.
6. Construct a regular grammar that generates the language accepted by this finite state automaton.



7. Construct a regular grammar that generates the language accepted by this finite state automaton.



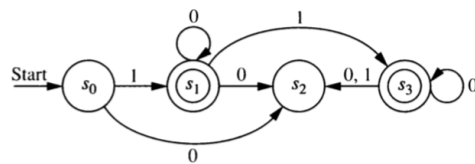
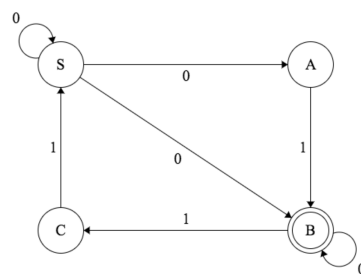
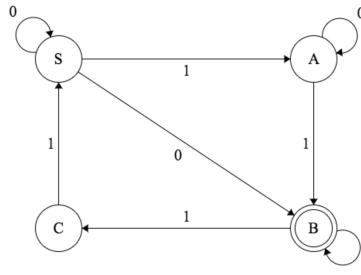
8. Construct a regular grammar that generates the language accepted by this finite state automaton.



9. In this problem you will show that any DFA or NFA with a lonely start state is equivalent to a regular grammar by showing how to construct a grammar G from such an automaton.
 - a. What will the terminals and nonterminals in your grammar be?
 - b. What will the start symbol be?
 - c. How are the production rules created?
 - d. Why was it necessary to have a lonely start state?

5.3 Some Review

- Convert each of the following NFA's into an equivalent DFA. (Remember: equivalent means that they recognize the same language.) Do this using our general algorithm for this task. For the first two, the start state is S .



6 Regular Languages

6.1 Equivalent Definitions

We have been working toward a result that says the following are all equivalent.

1. L can be described by a regular expression.
2. L is recognized by a DFA.
3. L is recognized by an NFA.
4. L is generated by a regular grammar.

The proof that these are equivalent amounts to describing algorithms that can convert from one representation to another. In particular we have shown

- a. $1 \Rightarrow 3$
- b. $2 \Rightarrow 3$
- c. $3 \Rightarrow 2$
- d. $2 \Rightarrow 4$
- e. $4 \Rightarrow 3$

Still missing from our list is

- f. $2 \Rightarrow 1$

Once we have that, we will see that all 4 definitions are equivalent.

Your Mission (You must choose to accept it)

For each conversion a – e, write a brief description of the algorithm. Your description should include the most important “big idea” and also any potential “gotchas”, things you have to be careful about or might forget about.

Bonus: Can you figure out how to do conversion f?

6.2 A language that is not regular

Since we now have several equivalent ways to show that a language is regular, we also have several ways to show that a language is not regular.

Here is a language that is not regular: $L = \{0^n 1^n \mid n = 0, 1, 2, \dots\}$.

1. What are our options for showing the language is not regular?
2. Which do you think will be easiest?
3. See if you can give a convincing argument that L is not regular.
4. What other languages can you construct that are not regular using the same basic idea?

6.3 Solutions

Compare your descriptions to the descriptions below. Did you leave out anything important? Do the solutions below leave out anything important?

1a. regex $\rightarrow$ NFA:

- 6 parts:
 - 3 base cases – pretty easy
 - 3 that show union, concatenation and Kleene star – these are the “interesting” part
- union: $A \cup B$ where $A = L(N_1)$ and $B = L(N_2)$.
 - keep all states and transitions from N_1 and N_2
 - add one new start state S ; S is final if either start state of N_1 or start state of N_2 is final
 - add some additional transitions
 - * if $S_1 \xrightarrow{x} A$, add $S \xrightarrow{x} A$
 - * if $S_2 \xrightarrow{x} B$, add $S \xrightarrow{x} B$
- concatenation: AB where $A = L(N_1)$ and $B = L(N_2)$.
 - keep all states and transitions from N_1 and N_2
 - start state from N_1 is the start state
 - only final states from N_2 are final
 - add some additional transitions
 - * if $A \xrightarrow{x} F$ (a final state in N_1), then add $A \xrightarrow{x} S_2$ (start state in N_2)
- Kleene star: $A = L(N_1)$
 - keep all states and transitions from N_1
 - add a new start state that is a final state (because $\lambda \in A^*$)
 - add new transitions from new start state (S)
 - * if $S_1 \xrightarrow{x} A$, add $S \xrightarrow{x} A$ (includes case where $A = S_1$)
 - add new transitions to old start state (S_1)
 - * if $A \xrightarrow{x} F$, where F is final, add $A \xrightarrow{x} S_1$
 - * If we are at the end of a string in $L(N_1)$, this lets us “restart”

1b. DFA $\rightarrow$ NFA

- easy: a DFA “is” an NFA

1c. NFA $\rightarrow$ DFA

- DFA states are **subsets of NFA states** (think: magnets on white board)
- A is final in DFA if A contains a final state of NFA
- Remember that $\{\}$ may be needed as one of the states in the DFA

1d. DFA (or NFA) $\rightarrow$ reg grammar

- $A \xrightarrow{x} B$ becomes
 - $A \rightarrow xB$
 - also $A \rightarrow x$ if B is final state
- If S is final in NFA/DFA, then add $S \rightarrow \lambda$

- Number of rules = number of transitions + number of transitions to final states + 1 more if the start state is final

1e. reg grammar $\rightarrow$ NFA

- states: non-terminals of grammar + one additional state F (a final state)
 - Start symbol is also start state
 - If $S \rightarrow \lambda$ is a rule, then S is a final state
- production rules become transitions in NFA
 - $A \rightarrow xB$ becomes $A \xrightarrow{x} B$
 - $A \rightarrow x$ becomes $A \xrightarrow{x} F$ (the added final state)

7 DFA/NFA to Regular Expression

7.1 GNFA (Generalied NFA)

A GNFA (Generalized NFA) is like an NFA but the edges may be labeled with any regular expression. One way of obtaining a regular expression from a DFA or NFA uses an algorithm that works with GNFA's.

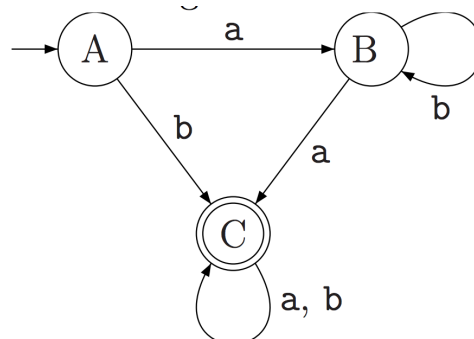
7.2 Algorithm for converting DFA/NFA to Regular Expression

Suppose we want to find an equivalent regular expression for some DFA or NFA. Here is an algorithm to do so.

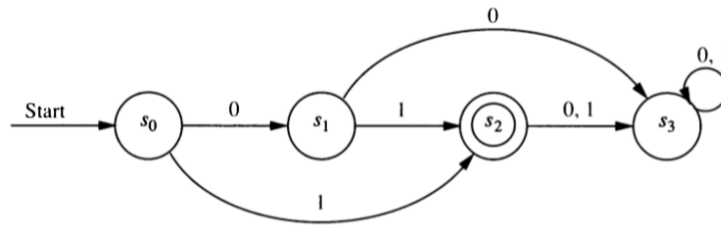
1. Modify the the original machine.
 - a. Add a **new start state** with a λ transition to the old start state. (We can skip this step if the start state is already lonely (has in-degree 0).)
 - b. Add a **new final state** with a λ transition from all old final states. (We can skip this step if there is exactly one final state and it has out-degree 0.)
 - c. The new final state will be the **only final state**.
 - d. **Replace mutiple edges** between any pair of states A and B with one edge labeled with the union of the labels of the original edges. [Example $A \xrightarrow{0,1} B$ becomes $A \xrightarrow{0 \cup 1} B$.]
2. Pick an internal state (not the start state or the final state) to “rip out”. Let’s call that state R .
 - a. If A doesn’t have a self-loop, replace every $A \xrightarrow{r_{in}} R \xrightarrow{r_{out}} B$ with $A \xrightarrow{(r_{in} r_{out})} B$.
 - b. If A has a self-loop labeled r_{self} , replace every $A \xrightarrow{r_{in}} R \xrightarrow{r_{out}} B$ with $A \xrightarrow{(r_{in} r_{self}^* r_{out})} B$.
 - c. If this results in multiple edges between two states A and B , replace them with one edge labeled with the union of their labels.
3. Repeat step 2 until the only states left are the start state and the final state. There will be only one transition remaining, and its label will be a regular expression for the language recognized by the original DFA or NFA.

7.3 Examples

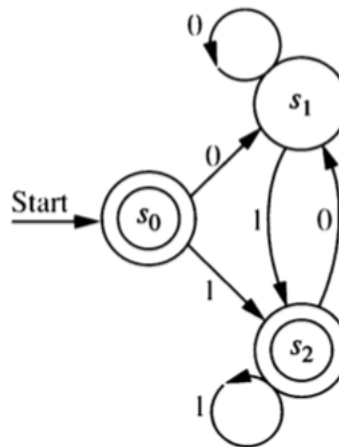
1. Systematically convert this DFA into an equivalent regular expression.



2. Systematically convert this DFA into an equivalent regular expression.



3. Systematically convert this DFA into an equivalent regular expression.



You can find additional worked examples online at places like

- <https://courses.cs.washington.edu/courses/cse311/14sp/kleene.pdf>
- https://courses.engr.illinois.edu/cs374/sp2019/extra_notes/01_nfa_to_reg.pdf

But note that sometimes the notation isn't quite the same.

- ε is sometimes used for what we have called λ .
- $+$ is sometimes used in place of $\cup$.

7.4 Review problems

1. Show that if A is a regular language, then $\overline{A}$ is also a regular language.
2. Show that if A and B are regular languages, then $A \cap B$ is also a regular language.
3. Convert the machines on this sheet to equivalent regular grammars.

8 Regular Expression in Python

8.1 Getting your python environment working

To experiment with regular expressions in python, you will need access to python 3 and the `requests` module. If you don't already have python running on your machine, here are some options.

8.1.1 repl.it

This python-in-a-browser seems able to do what we need (based on minimal testing)

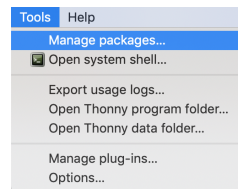
<https://repl.it/languages/python3>

8.1.2 Thonny

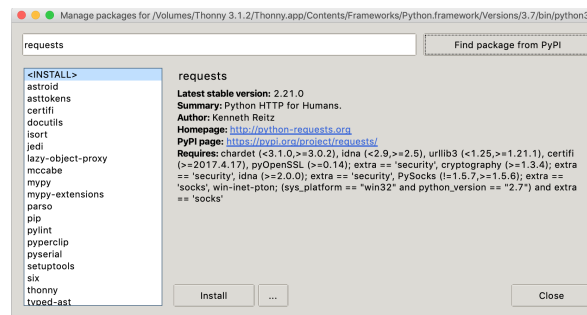
If you prefer to install a simple versions of python locally, try Thonny.

1. Download and install Thonny from <https://thonny.org/>.
2. Make sure the `requests` module is available.

In Thonny, you can install packages from the **tools > manage packages** menu:



Then search PyPI for `requests` and hit install.



8.1.3 Testing your python setup

If you can run the following and get the same output, you should be OK.

```
import re
import requests
# Download St. Augustine's confessions from CCEL and print it
url = 'https://www.ccel.org/ccel/augustine/confess.txt'
r = requests.get(url)
book = r.text
print(len(book))
```

```
## 708873
```

```
print(book[1015:1120])
```

```
##      Everywhere God wholly filleth all things, but neither heaven nor
##      Earth containeth him.
##
```

```
foo = re.compile("aug")
print(foo.search(book))
```

```
## <_sre.SRE_Match object; span=(4314, 4317), match='aug'>
```